General Remarks:

- Make sure the code compiles and run independent of Julia's `REPL` state
- Hand-in a PDF for the plots together with the code in a zip file (no binaries).
- Respect the academic code of honor. Teams have to work on their own. Detected plagiarism results in 0 points.
- Please contact the instructor for help and utilize the Olat forum for questions.

Radiosity (11 points)

Simulation Framework

In this assignment, distribution of light will be computed in a small 3D scene setup. A prepared [1]julia[2] project framework provides the 3D rendering and pre-computed matrices required for the light computation. The setup provides an application scenario to study different linear equation system solvers. Julia comes with an in-build solver (operator \), which is here utilized to check on correctness and to compare results.

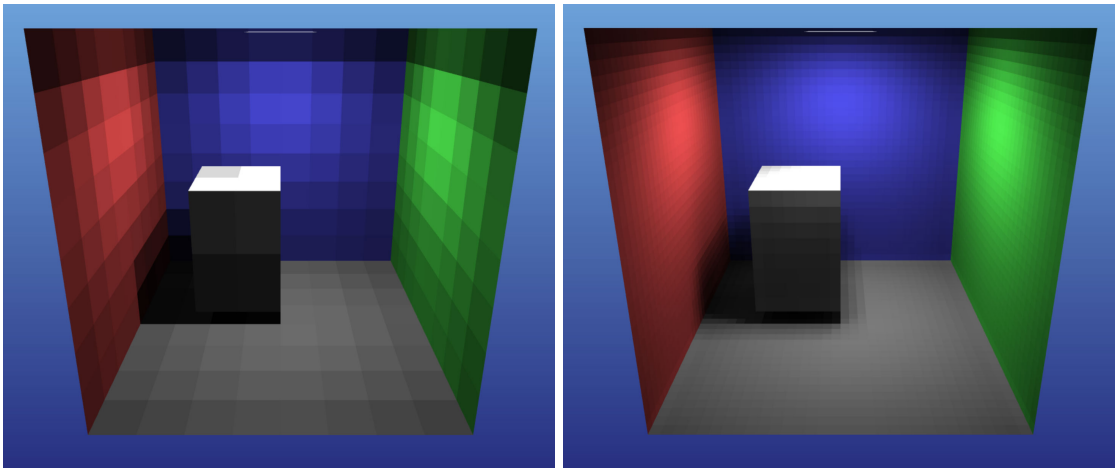

Figure 1: Scene showing simulated light emitted by an area light at the ceiling; (Left:) low resolution rectangular patches ($r = 8$), (Right:) high resolution ($r = 32$).

The 3D scene comprises of many small rectangular patches. For each patch the light intensity is computed by solving a linear system of equations; each line representing one patch:

$$x_i = b_i + \rho_i \sum_{j=1}^{n} F_{ij} x_j \tag{1}$$

with x_i the radiosity of patch i, b_i the self-emission, ρ_i the diffuse reflection coefficient (albedo), and F_{ij} the form factors describing the contribution of radiosity coming from

[1] https://julialang.org/downloads
[2] https://docs.julialang.org/en/v1/base/base

patch j on patch i; in matrix form:

$$\begin{bmatrix} 1 & -\rho F_{1,2} & -\rho F_{1,3} & \cdots & -\rho F_{1,n} \\ -\rho F_{2,1} & 1 & -\rho F_{2,3} & \cdots & -\rho F_{2,n} \\ \vdots & & \ddots & \vdots & \vdots \\ -\rho F_{n,1} & -\rho F_{n,2} & -\rho F_{n,3} & \cdots & 1 \end{bmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix} \tag{2}$$

with

$$F_{ij} = \frac{\cos\varphi_i \cos\varphi_j A_j}{\pi d_{ij}^2} \tag{3}$$

This formulation of the form factors F_{ij} is a rough approximation of a more precise integral over the hemisphere. The involved components are illustrated below:

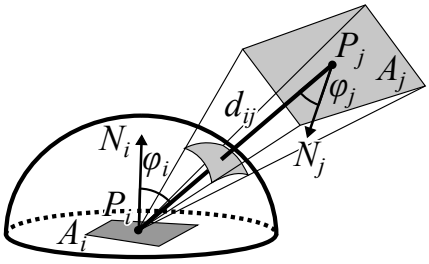

Figure 2: Related patches i and j as well as the necessary components to compute the form factor F_{ij}: patch positions P_*, areas A_*, angles φ_* between the center axis and the patch normals N_*.

The provided framework creates the scene dependent on a geometric resolution and precomputes the form factors for all patches via the helper function `createSceneEssentials( res::Int64, emission::Float64 )`. It returns the system matrix of equation (2) as well as the emissions per face (vector $\mathbf{b}$), a vertex list of the scene, a face list, and face colors.

The scene is displayed by generated `WebGL` via the helper function (see `Radiocity.jl`)

```
showMetaMeshFaceColors( Verts::Vector{Point3{Float64}},
                        Faces::Vector{QuadFace{Int64}},
                        FColors::Vector{RGB{Float64}} );
```

taking a vertex list, a face list and a color list (per face), respectively. Note that `julia` is a strongly typed language, but deduces the type automatically during run-time when they are omitted. The semicolon at the end optionally suppresses printed output in the `REPL`. When returning multiple values the last line in the function defines the return values; separated by a comma; e,g:

```
function solveGroundTruth( A::Matrix, b::Vector )
    A \ b, [0.0]
end
```

which returns a Matrix and an one element sized vector containing 0.0. The function, including an output of a time measurement, can be called as follows:

```
@time XGT, RGT = solveGroundTruth( Fij, Emission );
```

where `XGT` are the solved values of the equation system and `RGT` is a placeholder for a vector of the residual norm per solver iteration.

The task is to compute the vector $\mathbf{x}$ of equation (2) using different numerical methods for solving systems of linear equations.

Setup and Running the Framework

Julia provides a `REPL` to execute code in a current state. Install a local instance of julia on your system and make its binary available in the command-line; e.g.: linux:

```
wget https://julialang-s3.julialang.org/bin/linux/x64/1.6/...
        .../julia-1.6.3-linux-x86_64.tar.gz
tar -xvzf julia-1.6.3-linux-x86_64.tar.gz
export PATH=$PATH:/home/cXXX/julia-1.6.3/bin      <- e.g. in .bashrc
```

Navigate to your assignment directory via a terminal. Start the `REPL`. Note that for the first time the `MeshCat` package needs to be installed (']' within the `REPL` or via the `Pkg` in the script). The script is prepared such that it installs them on its first invocation. Thereafter, the related lines should be commented, to speed up the later executions; e.g.:

```
cd /home/cXXX/OptiNumAssignment_N2                <- in the terminal
julia
include("ExerciseAN2.jl")                         <- in the REPL
```

Then comment the line `install_packages()` and execute the script again in the `REPL`. The `REPL` provides its own state. You can inspect the actual matrices and live-code in the `REPL`.

You may want to setup an `IDE` with julia; e.g. vs code with the related extension. Then you can execute selected lines of the source files in the `REPL` by a press of a button (e.g. `ctrl-Enter`). Also note, when working on the solver part the form factors don't need to be recomputed and the cached F_{ij} in the `REPL` can be reused; i.e. calling solver and rendering functions only.

Tasks

The provided framework consists of three files: `ExerciseAN2.jl`, `ExerciseAN2_LGS.jl`, and `Radiosity.jl`. The latter provides the functionality to compute the form factors, to set up, and to display the scene. Additional code should go into the `ExerciseAN2*.jl` files, where `ExerciseAN2.jl` contains the main controls and `ExerciseAN2_LGS.jl` the definition of the solve and evaluate functions. Before handing-in make sure, that the code can also be executed from a terminal via '`julia ExerciseAN2.jl`' to ensure, that a correct computation is not dependent on an actual (maybe different) `REPL` state. Please use a geometry resolution of 8 to create the measurements and image comparisons, such that they are comparable over all programming groups. Also, include the system information when doing the time measurements (CPU).

(1) (3 Points) Solve the system of equations by using julia's default solver for a ground truth. Wrap the solver call in a function similar to `solveGroundTruth()` above. Use the resulting radiosity vector ($\mathbf{x}$) to scale the face colors of the scene by element-wise multiplication using the function `multRGB(a::RGB, b::Float64)` provided in `Radiosity.jl`.
Next, implement the iterative Jacobi solver. Therefore, introduce the `function jacobi(A::Matrix, b::Vector, maxiter = 500, epsilon = 1e-8)` in `ExerciseAN2_LGS.jl` returning the result radiosity vector $\mathbf{x}$ and a vector of the residual norms (one float per iteration step). The defaulted argument `maxiter` shall limit the iteration and `epsilon` defines the break criterion; if the current residual norm becomes smaller or equal epsilon - break the iteration.

(2) (2 Points) Implement the iterative methods Gauß-Seidel and successive over relaxation (SOR) using the same function signature and iteration control as for the `jacobi`. Employ $\omega = 1.25$ for the SOR method.

(3) (3 Points) Next, implement the methods gradient descent and conjugate gradients, again following the previous function signatures.

(4) (1 Point) Create two plots to compare the methods with your favorite plotting tool. One plot shall show the residual errors (ordinate) over the iteration count (abscissa); one line per method. Further, add an SOR run employing $\omega = 1.8$. The second plot shall compare computation times of the methods (bar-, box-, or violin-plot). Interpret the results (a few sentences).

(5) (2 Point) Compute the absolute difference of the method results and the ground truth. Use the difference to multiply the face colors for visualization. Show plots of three different solver iterations: $I = 3, 8, \max$. Note, that an additional over-scaling of the difference will be necessary to create read-able images. Provide the scaling in the plots (e.g. in the caption). What regions, i.e. which geometry patches, hold the largest errors for the different methods? Interpret the results (a few sentences).